

Foundations of Artificial Intelligence and Machine Learning

Models, Mathematics, and a Principled Taxonomy of Learning Paradigms

Er. Kumar Dhananjay
School of Computer Science Engineering and Technology
Bennett University, Greater Noida
`mr.kumar.dhananjay@gmail.com`

August 27, 2026

Abstract

This monograph presents a rigorous yet accessible treatment of machine learning as a mathematical discipline. It begins by locating machine learning inside the wider field of artificial intelligence, then formalises what a learning model actually is: a hypothesis space, a loss functional, an optimisation procedure, and a generalisation guarantee. The central contribution of the text is a precise account of *what* each learning paradigm assumes, *why* that assumption exists, and *how* a practitioner may distinguish one paradigm from another using an operational test based on the supervision signal. Supervised and unsupervised learning are then developed from first principles, with complete derivations for ordinary least squares, logistic regression, support vector machines, decision tree induction, backpropagation, Lloyd’s algorithm, expectation maximisation for Gaussian mixtures, and principal component analysis. Every method is accompanied by a worked numerical example computed by hand, and the exposition closes with four case studies drawn from network intrusion detection, medical image screening, industrial telemetry, and retail analytics.

Contents

1	Orientation: Artificial Intelligence, Machine Learning, Deep Learning	3
1.1	Three nested ideas	3
1.2	Why learn at all? The case for induction	3
2	What a Model Is: The Formal Anatomy of Learning	4
2.1	The four components	4
2.2	Risk, empirical risk, and the fundamental problem	4
2.3	The bias variance decomposition	5
3	The Taxonomy of Machine Learning: What, Why, How	5
3.1	The organising question	5
3.2	The five paradigms	5
3.3	How to distinguish them: an operational test	6
3.4	Supervised versus unsupervised: a side by side contrast	7
3.5	A second axis: task type within supervision	7
4	Supervised Learning: Mathematics and Derivations	7
4.1	Linear regression and ordinary least squares	7
4.2	Logistic regression: from regression to classification	9

4.3	Support vector machines: maximum margin classification	9
4.4	Decision trees and information gain	10
4.5	From a tree to an ensemble	11
4.6	Neural networks and backpropagation	12
4.7	Evaluating a supervised model	12
5	Unsupervised Learning: Mathematics and Derivations	13
5.1	k means clustering	13
5.2	Gaussian mixtures and expectation maximisation	14
5.3	Principal component analysis	15
5.4	Density based clustering and hierarchical clustering	16
5.5	Evaluating an unsupervised model	16
6	Case Studies	16
7	Model Selection, Validation and Common Errors	18
7.1	Cross validation	18
7.2	Eight errors that recur	18
7.3	The no free lunch principle	18
8	Summary	19

1 Orientation: Artificial Intelligence, Machine Learning, Deep Learning

1.1 Three nested ideas

Students routinely treat the phrases artificial intelligence, machine learning and deep learning as interchangeable. They are not. They denote three nested levels of ambition, and confusing them produces confused engineering.

Definition 1.1 (Artificial Intelligence). Artificial intelligence is the study and construction of computational systems that perform tasks which, when performed by humans, are said to require intelligence: perception, reasoning, planning, language, and action under uncertainty. The definition is task centred and is silent about mechanism. A hand written rule base that plays chess is artificial intelligence.

Definition 1.2 (Machine Learning). Machine learning is the subfield of artificial intelligence concerned with systems whose behaviour on a task improves automatically with experience. Following Mitchell, a program is said to learn from experience E with respect to a task class T and a performance measure P if its performance on tasks in T , as measured by P , improves with E . The definition is mechanism centred: the rules are induced from data rather than authored by a human.

Definition 1.3 (Deep Learning). Deep learning is the subfield of machine learning in which the hypothesis is a composition of many parameterised, differentiable transformations, so that representations of the input are learned hierarchically rather than engineered by hand.

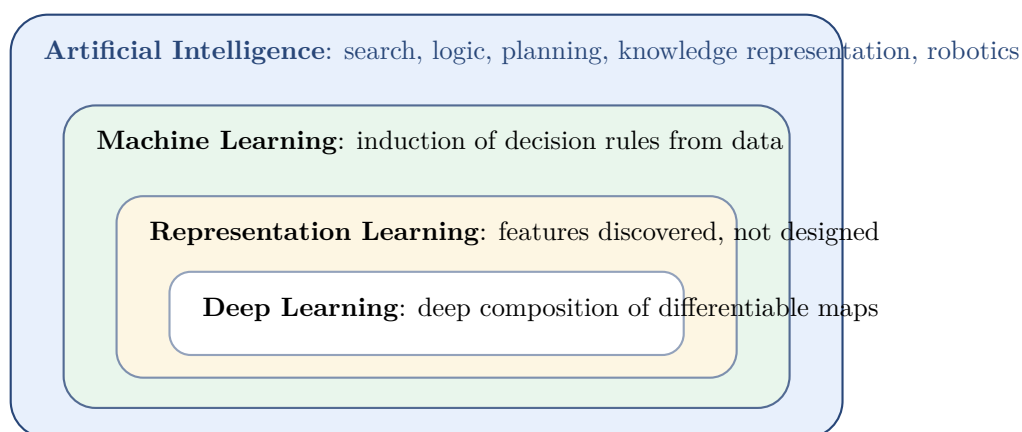


Figure 1: The nesting of fields. Each inner region inherits the goals of the outer region and adds a constraint on mechanism.

1.2 Why learn at all? The case for induction

Classical programming maps a specification to an algorithm. Learning is preferred exactly when one of four conditions holds.

1. **The rule is unknown to the expert.** A radiologist recognises a lesion but cannot write the pixel level predicate that fires on it. Tacit knowledge resists codification.
2. **The rule is known but astronomically large.** Enumerating every legal spam message is infeasible; the decision boundary is easier to approximate than to enumerate.
3. **The rule drifts.** Attack signatures, fraud patterns and consumer taste all change. A learned model can be refitted; a hand written rule base must be rewritten.

4. **The rule must be personalised.** One model per user is impossible to author manually and trivial to fit automatically.

Remark 1.1. The converse is equally important. When the mapping is exactly specifiable, deterministic and stable, learning is the wrong tool. Nobody should train a network to compute a checksum. A recurring failure in industry is the application of statistical induction to problems that already possess a closed form solution.

2 What a Model Is: The Formal Anatomy of Learning

2.1 The four components

Every supervised or unsupervised learner, from a two variable regression to a transformer with a trillion parameters, is assembled from exactly four components. Recognising them turns an intimidating zoo of algorithms into a small set of design decisions.

Component	Symbol and meaning	Design question it answers
Hypothesis space	$\mathcal{H} = \{f_\theta : \mathcal{X} \rightarrow \mathcal{Y} \mid \theta \in \Theta\}$	What shapes of decision rule am I willing to consider?
Loss function	$\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$	What exactly counts as a mistake, and how badly?
Optimiser	$\theta^* \in \arg \min_{\theta} \widehat{R}(f_\theta)$	How do I search Θ within my compute budget?
Generalisation control	regularisation, validation, early stopping	How do I ensure the fit transfers to unseen data?

Table 1: The four components of any learning model.

2.2 Risk, empirical risk, and the fundamental problem

Let $\mathcal{X}$ be the input space and $\mathcal{Y}$ the output space. Assume a fixed but unknown joint distribution $\mathbb{P}$ over $\mathcal{X} \times \mathcal{Y}$. The quantity we actually care about is the *true risk* or expected loss,

$$R(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathbb{P}}[\ell(f(\mathbf{x}), y)] = \int_{\mathcal{X} \times \mathcal{Y}} \ell(f(\mathbf{x}), y) d\mathbb{P}(\mathbf{x}, y). \quad (1)$$

Since $\mathbb{P}$ is unknown, $R(f)$ is not computable. What we possess is a sample $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ drawn independently from $\mathbb{P}$, and therefore only the *empirical risk*

$$\widehat{R}(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(\mathbf{x}_i), y_i). \quad (2)$$

Definition 2.1 (Empirical Risk Minimisation). The learning principle known as empirical risk minimisation selects $\hat{f} \in \arg \min_{f \in \mathcal{H}} \widehat{R}(f)$ and hopes that $R(\hat{f})$ is close to $\min_{f \in \mathcal{H}} R(f)$.

The entire theory of machine learning is the study of when that hope is justified. The excess risk decomposes as

$$\underbrace{R(\hat{f}) - R(f^*)}_{\text{excess risk}} = \underbrace{\left(R(\hat{f}) - \inf_{f \in \mathcal{H}} R(f)\right)}_{\text{estimation error}} + \underbrace{\left(\inf_{f \in \mathcal{H}} R(f) - R(f^*)\right)}_{\text{approximation error}}, \quad (3)$$

where f^* is the Bayes optimal predictor over all measurable functions. Enlarging $\mathcal{H}$ decreases approximation error and increases estimation error. This single tension explains model selection, regularisation, and the whole practice of validation.

2.3 The bias variance decomposition

For squared loss the tension becomes an exact identity. Let $y = f^*(\mathbf{x}) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$ and $\text{Var}(\varepsilon) = \sigma^2$, and let $\hat{f}$ be trained on a random sample. Fix a test point $\mathbf{x}_0$ and abbreviate $\bar{f}(\mathbf{x}_0) = \mathbb{E}_{\mathcal{D}}[\hat{f}(\mathbf{x}_0)]$.

Theorem 2.1 (Bias variance decomposition).

$$\mathbb{E}_{\mathcal{D}, \varepsilon}[(y_0 - \hat{f}(\mathbf{x}_0))^2] = \underbrace{(\bar{f}(\mathbf{x}_0) - f^*(\mathbf{x}_0))^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible}}. \quad (4)$$

Proof. Write $y_0 - \hat{f} = (y_0 - f^*) + (f^* - \bar{f}) + (\bar{f} - \hat{f})$, with all functions evaluated at $\mathbf{x}_0$. Square and take expectations. The first term has expectation σ^2 . The cross term between $(y_0 - f^*) = \varepsilon$ and the remaining two vanishes because ε is zero mean and independent of $\mathcal{D}$. The cross term between $(f^* - \bar{f})$, a constant with respect to $\mathcal{D}$, and $(\bar{f} - \hat{f})$ vanishes because $\mathbb{E}_{\mathcal{D}}[\bar{f} - \hat{f}] = 0$. The surviving squares are exactly the three named terms. $\square$

Remark 2.1. The third term is a floor. No model, however deep, can drive test error below σ^2 . A student reporting near zero test error on noisy data has almost certainly leaked labels into the training set.

3 The Taxonomy of Machine Learning: What, Why, How

3.1 The organising question

Machine learning paradigms are not distinguished by algorithm, by library, or by application. They are distinguished by **one question only**:

What form does the supervision signal take, and who supplies it?

Every other difference follows from the answer. This is why k means and Gaussian mixtures belong together even though their mathematics differ, and why regression and classification belong together even though their outputs differ.

3.2 The five paradigms

Paradigm	Data available	Objective being minimised	Canonical question answered
Supervised	$\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, every y_i given by an external annotator	$\frac{1}{n} \sum \ell(f(\mathbf{x}_i), y_i)$, a discrepancy against a known target	Given this new input, what is its label or value?
Unsupervised	$\{\mathbf{x}_i\}_{i=1}^n$, no targets at all	An internal criterion: compactness, likelihood, reconstruction error	What structure is present in this data?
Semi supervised	n_ℓ labelled and $n_u \gg n_\ell$ unlabelled points	Supervised loss plus a smoothness or consistency penalty on unlabelled data	Can cheap unlabelled data sharpen an expensive labelled fit?
Self supervised	$\{\mathbf{x}_i\}$ only; targets manufactured from the input itself	A supervised loss on a pretext task, for example predicting a masked token	What representation makes the input predictable from itself?

Paradigm	Data available	Objective	Question answered
Reinforcement	Trajectories of states, actions and scalar rewards, gathered by acting	Negative expected discounted return $-\mathbb{E}[\sum_t \gamma^t r_t]$	Which action sequence maximises long run reward?

Table 2: The five paradigms distinguished by their supervision signal.

3.3 How to distinguish them: an operational test

Rather than memorising the table, apply the following decision procedure to any problem placed before you. It terminates in at most four questions.

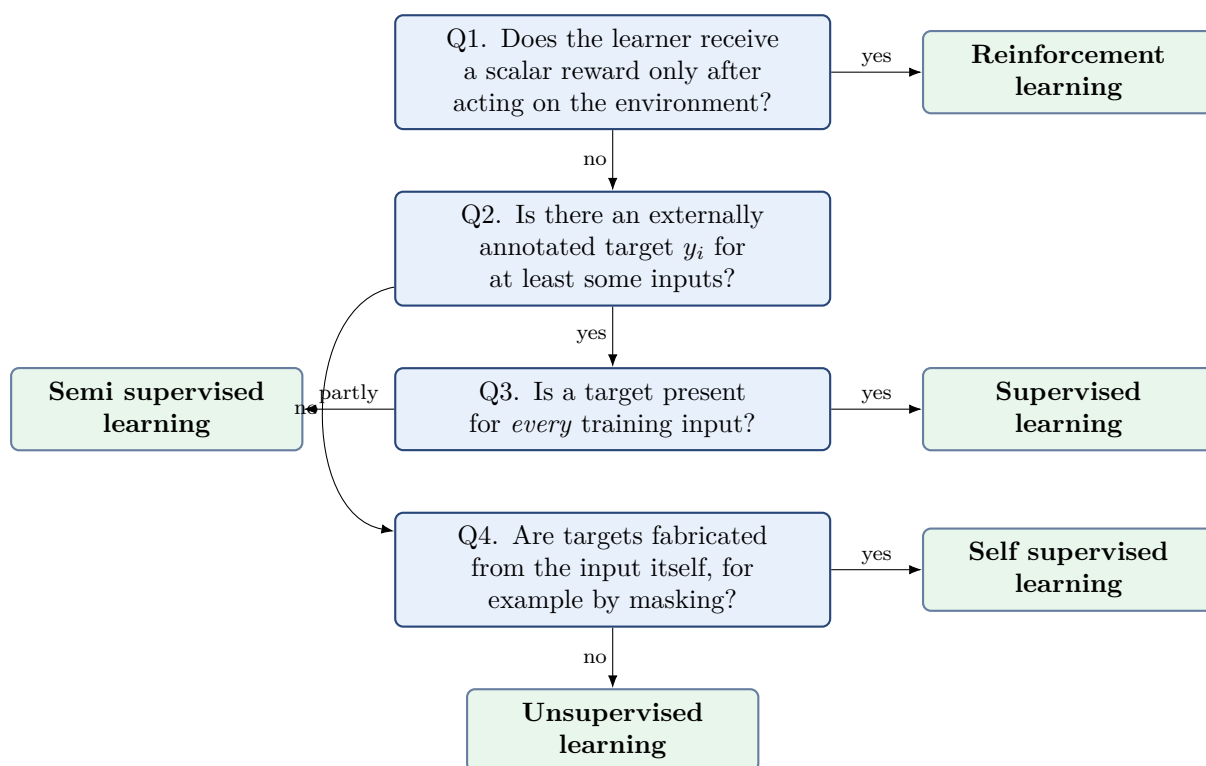


Figure 2: An operational test for paradigm identification. Note that the test inspects the *data*, never the algorithm.

Remark 3.1 (Three diagnostics that never fail). **First**, ask who paid for the labels. If a human annotator, a laboratory assay or a logged outcome supplied y , the setting is supervised. **Second**, ask whether the objective function can be evaluated on a single held out point without consulting a label. If it can, the method is unsupervised. **Third**, ask whether the ground truth is verifiable. A clustering has no correct answer to compare against; a classification does.

3.4 Supervised versus unsupervised: a side by side contrast

Dimension	Supervised learning	Unsupervised learning
Input to the learner	Pairs $(\mathbf{x}_i, y_i)$	Points $\mathbf{x}_i$ only
What is being estimated	Conditional law $\mathbb{P}(y \mathbf{x})$ or the regression function $\mathbb{E}[y \mathbf{x}]$	Marginal law $\mathbb{P}(\mathbf{x})$ or its low dimensional structure
Objective	External discrepancy against a target	Internal criterion: within cluster scatter, likelihood, reconstruction error
Ground truth for evaluation	Exists; held out accuracy is meaningful	Does not exist; only proxies such as silhouette or downstream utility
Typical failure mode	Overfitting the label noise	Discovering structure that is an artefact of scaling or of the chosen k
Cost driver	Annotation cost, often the dominant project expense	Compute and interpretation effort
Model selection	Cross validation against held out labels	Stability analysis, information criteria, expert review
Representative methods	Least squares, logistic regression, support vector machines, trees and forests, gradient boosting, neural networks	k means, Gaussian mixtures, hierarchical clustering, density based clustering, principal component analysis, autoencoders
Answer it gives	“This flow is an attack with probability 0.94.”	“These 8,200 flows form four coherent groups; group three is unlike the rest.”

Table 3: Supervised and unsupervised learning contrasted along eight dimensions.

3.5 A second axis: task type within supervision

Supervision answers *whether* a target exists. A second, independent axis answers *what kind* of target it is.

- **Classification:** $\mathcal{Y}$ is a finite unordered set. Loss is typically cross entropy or the zero one loss. Example: label a network flow as normal or as one of five attack families.
- **Regression:** $\mathcal{Y} = \mathbb{R}$ or $\mathbb{R}^d$. Loss is typically squared or absolute error. Example: predict remaining useful life of a bearing in hours.
- **Ordinal regression:** $\mathcal{Y}$ is finite but ordered, so confusing adjacent grades should cost less than confusing distant ones. Example: retinopathy severity on a five point scale.
- **Structured prediction:** $\mathcal{Y}$ is a combinatorial object such as a sequence, tree or segmentation mask.

Similarly, unsupervised tasks split into *clustering* (partition the sample), *dimensionality reduction* (compress the coordinates), *density estimation* (model $\mathbb{P}(\mathbf{x})$), and *association rule mining* (find co occurrence patterns).

4 Supervised Learning: Mathematics and Derivations

4.1 Linear regression and ordinary least squares

What. Model the conditional mean as an affine function of the inputs: $\mathbb{E}[y | \mathbf{x}] = \beta_0 + \sum_{j=1}^d \beta_j x_j$.

Why. It is the unique estimator that is unbiased and of minimum variance among all linear unbiased estimators under the Gauss Markov conditions, it has a closed form solution, and its coefficients are directly interpretable as marginal effects.

How. Collect the data into a design matrix $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$ whose first column is all ones, and a response vector $\mathbf{y} \in \mathbb{R}^n$. The objective is

$$J(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2 = (\mathbf{y} - \mathbf{X}\beta)^\top (\mathbf{y} - \mathbf{X}\beta). \quad (5)$$

Expanding,

$$J(\beta) = \mathbf{y}^\top \mathbf{y} - 2\beta^\top \mathbf{X}^\top \mathbf{y} + \beta^\top \mathbf{X}^\top \mathbf{X} \beta. \quad (6)$$

Differentiating with respect to β and using $\partial(\mathbf{a}^\top \beta)/\partial \beta = \mathbf{a}$ and $\partial(\beta^\top \mathbf{A} \beta)/\partial \beta = 2\mathbf{A}\beta$ for symmetric $\mathbf{A}$,

$$\nabla_\beta J = -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X} \beta \stackrel{!}{=} \mathbf{0} \implies \boxed{\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}} \quad (7)$$

The Hessian is $2\mathbf{X}^\top \mathbf{X} \succeq 0$, so the stationary point is a global minimum, and it is unique whenever $\mathbf{X}$ has full column rank.

Geometry. The fitted values are $\hat{\mathbf{y}} = \mathbf{X}\hat{\beta} = \mathbf{H}\mathbf{y}$ with $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$. Since $\mathbf{H}$ is idempotent and symmetric, it is the orthogonal projector onto the column space of $\mathbf{X}$. Least squares is therefore not an approximation heuristic; it is exactly orthogonal projection of the response onto the space of achievable predictions.

Regularisation. When $\mathbf{X}^\top \mathbf{X}$ is ill conditioned, add a penalty:

$$\text{Ridge: } \hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2 = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}, \quad (8)$$

$$\text{Lasso: } \hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1. \quad (9)$$

Ridge shrinks coefficients smoothly and always keeps them nonzero. Lasso produces exact zeros because the ℓ_1 ball has corners on the coordinate axes, which is why it doubles as a feature selector.

Example 4.1 (Least squares computed by hand). Four observations of intrusion detection alert volume y against traffic load x in gigabytes:

$$(x, y) \in \{(1, 2), (2, 4), (3, 5), (4, 8)\}.$$

Using the scalar form $\hat{\beta}_1 = S_{xy}/S_{xx}$ with $\bar{x} = 2.5$, $\bar{y} = 4.75$:

$$\begin{aligned} S_{xy} &= \sum (x_i - \bar{x})(y_i - \bar{y}) = (-1.5)(-2.75) + (-0.5)(-0.75) + (0.5)(0.25) + (1.5)(3.25) \\ &= 4.125 + 0.375 + 0.125 + 4.875 = 9.5, \\ S_{xx} &= 2.25 + 0.25 + 0.25 + 2.25 = 5.0. \end{aligned}$$

Hence $\hat{\beta}_1 = 9.5/5.0 = 1.9$ and $\hat{\beta}_0 = 4.75 - 1.9(2.5) = 0.0$. The fitted line is $\hat{y} = 1.9x$. Residuals are 0.1, 0.2, -0.7, 0.4, giving a residual sum of squares of $0.01 + 0.04 + 0.49 + 0.16 = 0.70$. With $S_{yy} = 7.5625 + 0.5625 + 0.0625 + 10.5625 = 18.75$, the coefficient of determination is $R^2 = 1 - 0.70/18.75 = 0.9627$. Roughly 96 percent of the variation in alert volume is explained by traffic load.

4.2 Logistic regression: from regression to classification

What. Model the log odds of the positive class as a linear function:

$$\log \frac{p(\mathbf{x})}{1-p(\mathbf{x})} = \mathbf{w}^\top \mathbf{x} + b, \quad p(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}. \quad (10)$$

Why. Fitting a straight line to a binary response is incoherent: predictions escape $[0, 1]$ and squared loss is not the correct likelihood for Bernoulli data. The logistic link is the canonical link for the Bernoulli family, which makes the resulting log likelihood concave and the estimator well behaved.

How. With $y_i \in \{0, 1\}$ the likelihood is $\prod_i p_i^{y_i} (1 - p_i)^{1-y_i}$, so the negative log likelihood, known as binary cross entropy, is

$$\mathcal{L}(\mathbf{w}, b) = - \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \quad (11)$$

Using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ and writing $z_i = \mathbf{w}^\top \mathbf{x}_i + b$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = - \sum_i \left[\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \right] p_i(1 - p_i) \mathbf{x}_i = \sum_{i=1}^n (p_i - y_i) \mathbf{x}_i = \mathbf{X}^\top (\mathbf{p} - \mathbf{y}). \quad (12)$$

The gradient is the design matrix applied to the prediction error, exactly as in least squares. No closed form root exists, so one iterates $\mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{X}^\top (\mathbf{p} - \mathbf{y})$, or uses Newton Raphson with Hessian $\mathbf{H} = \mathbf{X}^\top \mathbf{S} \mathbf{X}$ where $\mathbf{S} = \text{diag}(p_i(1 - p_i))$, which yields the iteratively reweighted least squares algorithm. Since $\mathbf{S} \succeq 0$, the Hessian is positive semidefinite and the objective is convex, so any local optimum is global.

Example 4.2 (One gradient step by hand). Two training flows with a single feature: $(x_1, y_1) = (2, 1)$ and $(x_2, y_2) = (-1, 0)$. Start at $w = 0$, $b = 0$, learning rate $\eta = 0.5$. Then $z_1 = z_2 = 0$, so $p_1 = p_2 = 0.5$. The gradient is $\partial \mathcal{L} / \partial w = (0.5 - 1)(2) + (0.5 - 0)(-1) = -1.0 - 0.5 = -1.5$, and $\partial \mathcal{L} / \partial b = (0.5 - 1) + (0.5 - 0) = 0$. The update gives $w \leftarrow 0 - 0.5(-1.5) = 0.75$, b unchanged. Now $z_1 = 1.5$, so $p_1 = 0.8176$, and $z_2 = -0.75$, so $p_2 = 0.3208$. The loss has fallen from $2 \times 0.6931 = 1.3863$ to $-\log(0.8176) - \log(0.6792) = 0.2013 + 0.3868 = 0.5881$. One step, and the model already separates the two points in the right direction.

4.3 Support vector machines: maximum margin classification

What. Among all separating hyperplanes, choose the one whose distance to the nearest training point is greatest.

Why. Infinitely many hyperplanes may separate a training set perfectly, yet they generalise differently. Margin maximisation is a principled tie breaker: bounds on generalisation error decrease as the margin grows, independently of the input dimension.

How. For labels $y_i \in \{-1, +1\}$, the geometric margin of point $\mathbf{x}_i$ is $y_i(\mathbf{w}^\top \mathbf{x}_i + b) / \|\mathbf{w}\|$. Fixing the functional margin of the closest point to unity gives the primal problem

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n. \quad (13)$$

Introducing multipliers $\alpha_i \geq 0$ yields the Lagrangian $\mathcal{J} = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_i \alpha_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1]$. Setting $\partial \mathcal{J} / \partial \mathbf{w} = \mathbf{0}$ and $\partial \mathcal{J} / \partial b = 0$ gives $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ and $\sum_i \alpha_i y_i = 0$. Substituting back produces the dual

$$\max_{\alpha \geq 0} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \quad \text{subject to} \quad \sum_i \alpha_i y_i = 0. \quad (14)$$

Karush Kuhn Tucker complementary slackness, $\alpha_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1] = 0$, forces $\alpha_i = 0$ for every point strictly outside the margin. Only the support vectors carry weight, which is the source of the method's name and of its memory efficiency.

Two extensions. Slack variables $\xi_i \geq 0$ with penalty $C \sum_i \xi_i$ handle nonseparable data. Because the dual depends on inputs only through inner products, replacing $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$ with a kernel $K(\mathbf{x}_i, \mathbf{x}_j)$ fits a linear model in an implicit high dimensional feature space at no extra cost. The radial basis kernel $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$ corresponds to an infinite dimensional space.

4.4 Decision trees and information gain

What. A tree recursively partitions the input space with axis aligned splits and predicts a constant in each leaf.

Why. Trees are invariant to monotone rescaling of features, handle mixed categorical and numeric inputs natively, capture interactions without explicit terms, and yield rules a domain expert can read. In security and clinical settings that auditability is often decisive.

How. Impurity of a node S with class proportions p_c is measured by entropy or by the Gini index:

$$H(S) = - \sum_{c=1}^C p_c \log_2 p_c, \quad G(S) = 1 - \sum_{c=1}^C p_c^2. \quad (15)$$

A candidate feature A partitions S into subsets $\{S_v\}$, and the information gain is

$$IG(S, A) = H(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} H(S_v). \quad (16)$$

The greedy induction algorithm selects at each node the feature of maximum gain, then recurses.

Algorithm 1 Greedy decision tree induction

```

1: function BUILD_TREE( $S$ , features  $F$ )
2:   if all labels in  $S$  are identical or  $F = \emptyset$  or stopping rule fires then
3:     return leaf predicting the majority class of  $S$ 
4:   end if
5:    $A^* \leftarrow \arg \max_{A \in F} IG(S, A)$ 
6:   create an internal node testing  $A^*$ 
7:   for all values  $v$  of  $A^*$  do
8:     attach BUILD_TREE( $S_v$ ,  $F \setminus \{A^*\}$ ) as the child for  $v$ 
9:   end for
10:  return the node
11: end function

```

Example 4.3 (Information gain on network flow records). Ten labelled flows, two categorical features:

Flow	Protocol	Duration	Label
1	TCP	Short	Attack
2	TCP	Short	Attack
3	TCP	Short	Attack
4	TCP	Long	Attack
5	TCP	Long	Attack
6	TCP	Long	Normal
7	UDP	Short	Attack
8	UDP	Short	Normal
9	UDP	Long	Normal
10	UDP	Long	Normal

The root contains six attacks and four normals, so

$$H(S) = -0.6 \log_2 0.6 - 0.4 \log_2 0.4 = 0.6(0.7370) + 0.4(1.3219) = 0.9710 \text{ bits.}$$

Split on Protocol. TCP holds six flows, five attacks and one normal; UDP holds four flows, one attack and three normals.

$$\begin{aligned} H(\text{TCP}) &= -\frac{5}{6} \log_2 \frac{5}{6} - \frac{1}{6} \log_2 \frac{1}{6} = 0.2192 + 0.4308 = 0.6500, \\ H(\text{UDP}) &= -\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4} = 0.5000 + 0.3113 = 0.8113, \\ \sum \frac{|S_v|}{|S|} H(S_v) &= 0.6(0.6500) + 0.4(0.8113) = 0.3900 + 0.3245 = 0.7145, \\ IG(S, \text{Protocol}) &= 0.9710 - 0.7145 = \mathbf{0.2565} \text{ bits.} \end{aligned}$$

Split on Duration. Short holds five flows, four attacks and one normal; Long holds five flows, two attacks and three normals.

$$\begin{aligned} H(\text{Short}) &= -0.8 \log_2 0.8 - 0.2 \log_2 0.2 = 0.2575 + 0.4644 = 0.7219, \\ H(\text{Long}) &= -0.4 \log_2 0.4 - 0.6 \log_2 0.6 = 0.5288 + 0.4422 = 0.9710, \\ \sum \frac{|S_v|}{|S|} H(S_v) &= 0.5(0.7219) + 0.5(0.9710) = 0.8465, \\ IG(S, \text{Duration}) &= 0.9710 - 0.8465 = \mathbf{0.1245} \text{ bits.} \end{aligned}$$

Protocol yields more than twice the gain, so it becomes the root test. The interpretation is concrete: knowing the protocol removes about 0.26 bits of the 0.97 bits of uncertainty about whether a flow is hostile.

Remark 4.1 (Why gain ratio exists). Raw information gain is biased towards features with many distinct values. A flow identifier would perfectly partition the data and score maximum gain while generalising not at all. The C4.5 correction divides by the split information $-\sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$, penalising high cardinality splits.

4.5 From a tree to an ensemble

A single deep tree has low bias and ruinous variance. Two remedies exploit the bias variance identity of Equation (4) directly.

- **Bagging and random forests** average B trees fitted on bootstrap resamples, each split restricted to a random subset of $m \approx \sqrt{d}$ features. If the individual trees have variance σ^2 and pairwise correlation ρ , the average has variance

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2, \quad (17)$$

which is why feature subsampling matters: it lowers ρ , and $\rho\sigma^2$ is the floor that no amount of extra trees can remove.

- **Boosting** fits trees sequentially, each on the gradient of the loss with respect to the current prediction. Gradient boosting attacks the bias term rather than the variance term, and consequently uses shallow trees where random forests use deep ones.

4.6 Neural networks and backpropagation

What. A feedforward network of depth L computes

$$\mathbf{a}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \phi(\mathbf{z}^{(l)}), \quad l = 1, \dots, L. \quad (18)$$

Why. The universal approximation property guarantees that a single hidden layer with enough units approximates any continuous function on a compact set to arbitrary accuracy. Depth matters because certain functions require exponentially many units in a shallow network but only polynomially many when composed across layers.

How. Let C be the loss and define the error signal $\delta^{(l)} = \partial C / \partial \mathbf{z}^{(l)}$. The chain rule gives the four equations of backpropagation:

$$\delta^{(L)} = \nabla_{\mathbf{a}^{(L)}} C \odot \phi'(\mathbf{z}^{(L)}), \quad (19)$$

$$\delta^{(l)} = \left((\mathbf{W}^{(l+1)})^\top \delta^{(l+1)} \right) \odot \phi'(\mathbf{z}^{(l)}), \quad (20)$$

$$\frac{\partial C}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top, \quad (21)$$

$$\frac{\partial C}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \quad (22)$$

Here $\odot$ is elementwise multiplication. Equation (20) is the whole algorithm in one line: error is propagated backwards by the transpose of the forward weight matrix, then modulated by the local derivative of the activation.

Remark 4.2 (Where vanishing gradients come from). Composing Equation (20) across L layers multiplies L factors of ϕ' . For the logistic sigmoid, $\max_z \phi'(z) = 0.25$, so a gradient reaching layer one is scaled by at most 4^{-L} . At $L = 10$ that is roughly 10^{-6} . This single observation motivates rectified activations, residual connections, and careful weight initialisation.

4.7 Evaluating a supervised model

For binary classification, arrange predictions into a confusion matrix and derive:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad (23)$$

$$\text{Recall (TPR)} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (24)$$

$$\text{FPR} = \frac{FP}{FP + TN}, \quad \text{Specificity} = \frac{TN}{TN + FP}. \quad (25)$$

A single balanced summary for imbalanced problems is the Matthews correlation coefficient,

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \in [-1, 1], \quad (26)$$

which returns zero for a random or constant predictor whatever the class proportions.

Example 4.4 (The accuracy paradox, quantified). An intrusion detector is evaluated on 10,000 flows of which 1,000 are hostile. It reports $TP = 920$, $FN = 80$, $FP = 150$, $TN = 8850$.

$$\text{Accuracy} = (920 + 8850)/10000 = 0.9770,$$

$$\text{Precision} = 920/1070 = 0.8598,$$

$$\text{Recall} = 920/1000 = 0.9200,$$

$$F_1 = \frac{2(0.8598)(0.9200)}{0.8598 + 0.9200} = \frac{1.5820}{1.7798} = 0.8889,$$

$$\text{FPR} = 150/9000 = 0.0167.$$

Now consider a trivial detector that labels every flow normal. Its accuracy is $9000/10000 = 0.90$, only seven points below the real model, yet its recall is zero and it is operationally worthless. Accuracy alone is uninformative whenever classes are imbalanced, which is the normal condition in security, fraud and clinical screening. Report precision, recall, F_1 and the area under the receiver operating characteristic curve, and state the operating threshold.

5 Unsupervised Learning: Mathematics and Derivations

5.1 k means clustering

What. Partition $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ into k disjoint clusters $C_1, \dots, C_k$ minimising within cluster scatter.

Why. It is the simplest formalisation of the intuition that a good group is one whose members resemble their own centre more than any other centre. It is fast, its output is interpretable as k prototypes, and it scales to millions of points.

How. The objective, the within cluster sum of squares, is

$$J(\{C_j\}, \{\boldsymbol{\mu}_j\}) = \sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2. \quad (27)$$

Minimising jointly over assignments and centroids is combinatorially hard, so Lloyd's algorithm alternates two steps, each of which is optimal given the other.

Proposition 5.1 (Monotone descent). *Holding centroids fixed, assigning each point to its nearest centroid cannot increase J , since each term is minimised independently. Holding assignments fixed, setting $\partial J / \partial \boldsymbol{\mu}_j = -2 \sum_{\mathbf{x}_i \in C_j} (\mathbf{x}_i - \boldsymbol{\mu}_j) = \mathbf{0}$ gives $\boldsymbol{\mu}_j = |C_j|^{-1} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i$, the cluster mean, which again cannot increase J . Therefore J is nonincreasing, and since only finitely many partitions exist, the algorithm terminates at a local minimum.*

Algorithm 2 Lloyd's algorithm for k means

- 1: initialise centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k$ (kmeans++ seeding recommended)
 - 2: **repeat**
 - 3: **Assignment:** $c_i \leftarrow \arg \min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2$ for every i
 - 4: **Update:** $\boldsymbol{\mu}_j \leftarrow \frac{1}{|C_j|} \sum_{i: c_i=j} \mathbf{x}_i$ for every j
 - 5: **until** no assignment changes
 - 6: **return** $\{C_j\}, \{\boldsymbol{\mu}_j\}$
-

Example 5.1 (Two iterations by hand, including a poor initialisation). Six two dimensional points:

$$P_1(1, 1), P_2(2, 1), P_3(1, 2), P_4(8, 8), P_5(9, 8), P_6(8, 9).$$

Deliberately choose a bad seeding: $\mu_1 = (1, 1)$, $\mu_2 = (2, 1)$.

Iteration 1, assignment. Squared distances give $P_1 \rightarrow C_1$ (0 against 1), $P_2 \rightarrow C_2$ (1 against 0), $P_3 \rightarrow C_1$ (1 against 2), and for the far cluster P_4 : $d_1^2 = 98$, $d_2^2 = 85$, so $P_4 \rightarrow C_2$; likewise $P_5 \rightarrow C_2$ (113 against 98) and $P_6 \rightarrow C_2$ (113 against 100). The partition is badly wrong: $C_1 = \{P_1, P_3\}$, $C_2 = \{P_2, P_4, P_5, P_6\}$.

Iteration 1, update. $\mu_1 = (1, 1.5)$ and $\mu_2 = \left(\frac{2+8+9+8}{4}, \frac{1+8+8+9}{4}\right) = (6.75, 6.5)$.

Iteration 2, assignment. Now $P_2(2, 1)$ has $d_1 = \sqrt{1 + 0.25} = 1.118$ against $d_2 = \sqrt{22.5625 + 30.25} = 7.267$, so it moves to C_1 . All far points stay in C_2 . The partition is now $C_1 = \{P_1, P_2, P_3\}$, $C_2 = \{P_4, P_5, P_6\}$.

Iteration 2, update. $\mu_1 = (4/3, 4/3) = (1.333, 1.333)$ and $\mu_2 = (25/3, 25/3) = (8.333, 8.333)$.

Iteration 3. No assignment changes, so the algorithm halts. The final objective is

$$J = (0.222 + 0.556 + 0.556) + (0.222 + 0.556 + 0.556) = \frac{8}{3} \approx 2.667.$$

Two lessons follow. First, the algorithm recovered from an adversarial seeding in two sweeps because the true clusters are well separated. Second, with clusters that overlap it would not have; this is precisely why kmeans++ seeding and multiple restarts are standard practice.

Remark 5.1 (Choosing k). Equation (27) decreases monotonically in k and reaches zero at $k = n$, so J cannot be used to select k . Use the elbow heuristic on J , the silhouette coefficient

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1],$$

where $a(i)$ is the mean distance to points in the same cluster and $b(i)$ the mean distance to the nearest other cluster, the gap statistic, or the Davies Bouldin index. Better still, validate k against a downstream decision.

5.2 Gaussian mixtures and expectation maximisation

What. Model the data as generated by a mixture of k Gaussian components,

$$p(\mathbf{x}) = \sum_{j=1}^k \pi_j \mathcal{N}(\mathbf{x} \mid \mu_j, \Sigma_j), \quad \pi_j \geq 0, \quad \sum_j \pi_j = 1. \quad (28)$$

Why. k means assigns each point to exactly one cluster and implicitly assumes spherical clusters of equal size. A mixture model relaxes both restrictions: it gives *soft* memberships and allows each component its own covariance, so elongated and unequally sized clusters are representable. It also gives a genuine density, which supports likelihood based anomaly scoring.

How. Direct maximisation of the log likelihood $\ell(\theta) = \sum_i \log \sum_j \pi_j \mathcal{N}(\mathbf{x}_i \mid \mu_j, \Sigma_j)$ is awkward because of the logarithm of a sum. Expectation maximisation introduces latent assignments z_i and alternates:

E step. Compute responsibilities

$$\gamma_{ij} = \mathbb{P}(z_i = j \mid \mathbf{x}_i, \theta^{(t)}) = \frac{\pi_j \mathcal{N}(\mathbf{x}_i \mid \mu_j, \Sigma_j)}{\sum_{m=1}^k \pi_m \mathcal{N}(\mathbf{x}_i \mid \mu_m, \Sigma_m)}. \quad (29)$$

M step. With $N_j = \sum_i \gamma_{ij}$, update in closed form

$$\pi_j = \frac{N_j}{n}, \quad \boldsymbol{\mu}_j = \frac{1}{N_j} \sum_i \gamma_{ij} \mathbf{x}_i, \quad \boldsymbol{\Sigma}_j = \frac{1}{N_j} \sum_i \gamma_{ij} (\mathbf{x}_i - \boldsymbol{\mu}_j)(\mathbf{x}_i - \boldsymbol{\mu}_j)^\top. \quad (30)$$

Each iteration increases the observed data log likelihood, because the algorithm maximises the evidence lower bound $\ell(\theta) \geq \mathbb{E}_q[\log p(\mathbf{x}, z | \theta)] + \mathbb{H}[q]$, with equality when q equals the posterior computed in the E step.

Remark 5.2 (k means as a limiting case). Fix $\boldsymbol{\Sigma}_j = \varepsilon \mathbf{I}$ and let $\varepsilon \rightarrow 0$. The responsibilities collapse to indicators of the nearest centroid, and expectation maximisation degenerates exactly into Lloyd's algorithm. k means is therefore hard assignment Gaussian mixture modelling with tied spherical covariance.

5.3 Principal component analysis

What. Find an orthonormal basis $\{\mathbf{v}_1, \dots, \mathbf{v}_m\}$, $m < d$, such that projecting the centred data onto its span retains maximum variance.

Why. High dimensional data is expensive to store, hard to visualise, and statistically treacherous: with d comparable to n , distances concentrate and estimators become unstable. Principal component analysis compresses while discarding the least informative directions, and it decorrelates the coordinates.

How. Centre the data, form the sample covariance $\mathbf{S} = \frac{1}{n-1} \mathbf{X}_c^\top \mathbf{X}_c$, and maximise the projected variance $\mathbf{v}^\top \mathbf{S} \mathbf{v}$ subject to $\mathbf{v}^\top \mathbf{v} = 1$. The Lagrangian is $\mathcal{J} = \mathbf{v}^\top \mathbf{S} \mathbf{v} - \lambda(\mathbf{v}^\top \mathbf{v} - 1)$, and

$$\frac{\partial \mathcal{J}}{\partial \mathbf{v}} = 2\mathbf{S} \mathbf{v} - 2\lambda \mathbf{v} = \mathbf{0} \implies \mathbf{S} \mathbf{v} = \lambda \mathbf{v}. \quad (31)$$

So the optimal direction is an eigenvector of $\mathbf{S}$, and since $\mathbf{v}^\top \mathbf{S} \mathbf{v} = \lambda$, the maximum variance direction is the eigenvector of the largest eigenvalue. Subsequent components are the remaining eigenvectors in decreasing order of eigenvalue, mutually orthogonal because $\mathbf{S}$ is symmetric. The fraction of variance retained by m components is $\sum_{j \leq m} \lambda_j / \sum_{j \leq d} \lambda_j$.

Example 5.2 (Principal components computed by hand). Four sensor readings (x_1, x_2) : $(2, 1), (4, 4), (6, 5), (8, 8)$. The means are $\bar{x}_1 = 5$, $\bar{x}_2 = 4.5$, so the centred data are $(-3, -3.5), (-1, -0.5), (1, 0.5), (3, 3.5)$. With $n - 1 = 3$,

$$\begin{aligned} s_{11} &= \frac{9+1+1+9}{3} = 6.6667, & s_{22} &= \frac{12.25+0.25+0.25+12.25}{3} = 8.3333, \\ s_{12} &= \frac{10.5+0.5+0.5+10.5}{3} = 7.3333, & \mathbf{S} &= \begin{pmatrix} 6.6667 & 7.3333 \\ 7.3333 & 8.3333 \end{pmatrix}. \end{aligned}$$

The characteristic equation is $\lambda^2 - \text{tr}(\mathbf{S})\lambda + \det(\mathbf{S}) = 0$ with $\text{tr} = 15.0$ and $\det = 6.6667(8.3333) - 7.3333^2 = 55.5556 - 53.7778 = 1.7778$. Hence

$$\lambda = \frac{15 \pm \sqrt{225 - 7.1111}}{2} = \frac{15 \pm 14.7611}{2} \implies \lambda_1 = 14.8806, \quad \lambda_2 = 0.1194.$$

The first component explains $14.8806/15.0 = 99.20$ percent of the total variance. Its eigenvector satisfies $(6.6667 - 14.8806)v_1 + 7.3333v_2 = 0$, so $v_2 = 1.1201v_1$, and after normalising

$$\mathbf{v}_1 = (0.6664, 0.7456)^\top.$$

Projecting onto $\mathbf{v}_1$ compresses two sensors into one derived index while losing less than one percent of the variability. That single number is what one would then feed to a downstream monitor.

5.4 Density based clustering and hierarchical clustering

- **DBSCAN** declares a point a core point if at least `minPts` neighbours lie within radius ϵ , then merges density reachable core points. Unlike k means it does not require k in advance, discovers arbitrarily shaped clusters, and labels low density points as noise, which makes it a natural anomaly detector. Its weakness is sensitivity to ϵ when densities vary across the data.
- **Agglomerative hierarchical clustering** begins with n singleton clusters and repeatedly merges the closest pair according to a linkage rule: single linkage uses the minimum pairwise distance and produces chained, elongated clusters; complete linkage uses the maximum and produces compact ones; Ward linkage merges the pair that least increases within cluster variance and is the closest hierarchical analogue of k means. The output is a dendrogram, from which any number of clusters can be read by cutting at a chosen height. No commitment to k is needed before seeing the structure.

5.5 Evaluating an unsupervised model

There is no held out accuracy, because there is no ground truth. Three families of evidence are used instead.

1. **Internal indices** measure geometry alone: silhouette coefficient, Davies Bouldin index, Calinski Harabasz index. They reward compact, well separated clusters, and they encode assumptions. The silhouette rewards spherical clusters, so it will quietly punish a correct DBSCAN solution on crescent shaped data.
2. **Stability** measures reproducibility: refit on bootstrap resamples and measure agreement with the adjusted Rand index. A structure that dissolves under resampling was an artefact.
3. **Extrinsic utility** is the only decisive test: does the discovered partition improve a downstream decision? Do the four discovered flow groups correspond to distinct analyst playbooks? Do the customer segments respond differently to a campaign?

6 Case Studies

Case Study 6.1 (Supervised: network intrusion detection). **Problem.** An enterprise mirrors traffic to a sensor that extracts per flow features: duration, protocol, byte counts in each direction, packet interarrival statistics, flag counts, and window sizes. Historical flows have been labelled by analysts as normal or as one of several attack families.

Why supervised. Labels exist and were paid for by analyst time, so the objective is a discrepancy against a known target. Diagnostic question one is satisfied immediately.

Pipeline. Standardise numeric features, since a byte count in the millions would otherwise dominate a flag count in the units. Encode categorical protocol as one hot indicators. Split by time, not at random: training on future flows and testing on past flows leaks information and inflates reported accuracy, a mistake that appears in a large fraction of published intrusion detection results. Address the roughly one hundred to one class imbalance with class weighting in the loss rather than naive oversampling, which duplicates points and encourages memorisation.

Model. A gradient boosted tree ensemble is a strong default: it handles mixed types, needs no scaling, and captures interactions such as “short duration *and* many SYN flags *and* no reply.” A calibrated probability output is essential because the alert threshold is an operational decision.

Evaluation. Report recall at a fixed and tolerable false positive rate, not accuracy. If the sensor processes ten million flows a day, a false positive rate of one percent generates one hundred thousand alerts and the security operations centre stops reading them. The correct headline metric is therefore something like “recall of 0.91 at a false positive rate of 10^{-4} ”.

Explainability. Shapley additive explanations attributing an alert to specific features convert a score into a triage aid. An analyst who is told *why* a flow was flagged closes the ticket in minutes rather than hours.

Case Study 6.2 (Unsupervised: discovering unknown attack behaviour). **Problem.** The same sensor, but the concern is now the attack that has no signature and therefore no label. A supervised model trained only on known families is by construction blind to a novel one.

Why unsupervised. No target exists for the phenomenon of interest. The objective must be internal: model what normal looks like, then score deviation.

Approach. Fit a density model or an autoencoder on traffic drawn from a period believed to be clean. For an autoencoder, the anomaly score is reconstruction error $\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2$; for a Gaussian mixture it is the negative log likelihood $-\log p(\mathbf{x})$. Set the alert threshold at a high quantile of the score on validation data, for example the 99.9th percentile, so that the alert budget is fixed in advance.

Honest limitations. The clean period assumption is load bearing and usually false; any attack present during training is learned as normal. Concept drift means the model must be refitted as legitimate traffic evolves, or the false positive rate will climb steadily. And a high anomaly score means unusual, not malicious. A backup job that runs once a quarter is extremely unusual and entirely benign.

The productive combination. In practice the two case studies compose. The unsupervised layer surfaces candidates, analysts label a small number of them, and those labels feed the supervised model. That loop is semi supervised learning implemented as an organisational process rather than as a single loss function.

Case Study 6.3 (Ordinal supervised: retinal image screening). **Problem.** Grade retinal fundus photographs on a five point severity scale from no disease to proliferative disease.

Why the task type matters. This is classification with an ordered label set. Treating it as plain multiclass classification throws away the ordering, so the model is penalised identically for confusing grade 0 with grade 1 and for confusing grade 0 with grade 4. Clinically those two errors are not remotely equivalent. Use an ordinal formulation, or a regression head with thresholds, and evaluate with quadratic weighted kappa, which penalises distant disagreements more heavily.

Data realities. Labels are expensive and graders disagree, so the irreducible term σ^2 in Equation (4) is substantial. Report inter grader agreement as a ceiling; a model that matches human to human agreement has reached the noise floor and further accuracy claims warrant scepticism. Where images cannot leave the institutions that hold them, a federated formulation trains locally and shares only model updates, which changes the optimisation but not the taxonomy: the task remains supervised.

Case Study 6.4 (Unsupervised then supervised: retail segmentation and churn). **Stage one, unsupervised.** Given transaction logs, compute recency, frequency and monetary value per customer, standardise, and cluster. Four segments emerge: dormant, occasional, loyal, and high value irregular. There is no ground truth here; the segments are validated by showing that they respond differently to a test campaign.

Stage two, supervised. Define churn as no purchase in ninety days, which manufactures a label from logged outcomes. Fit a classifier with segment membership as an additional input feature. The unsupervised stage has produced a compact, human meaningful feature that the supervised stage exploits.

The lesson. The paradigms are not competitors. Unsupervised methods generate structure and features; supervised methods convert structure into decisions.

7 Model Selection, Validation and Common Errors

7.1 Cross validation

Partition the data into K folds; for each fold, train on the remaining $K - 1$ and evaluate on the held out fold; average. Five fold and ten fold are the usual choices. With $K = n$ the procedure is leave one out: nearly unbiased, but of high variance and expensive.

Situation	Correct scheme	Reason
Independent samples	Stratified K fold	Preserves class proportions in every fold
Time ordered data	Rolling origin, expanding window	Prevents the future from informing the past
Grouped data, for example several images per patient	Grouped K fold	Stops the same subject appearing in train and test
Hyperparameter tuning	Nested cross validation	An inner loop tunes, an outer loop estimates honestly

Table 4: Matching the validation scheme to the data generating process.

7.2 Eight errors that recur

1. **Leakage through preprocessing.** Standardising, imputing or selecting features on the full dataset before splitting lets test statistics into training. Fit every transformer inside the training fold only.
2. **Leakage through time.** Random splits on temporal data almost always inflate results. Split chronologically.
3. **Optimising accuracy under imbalance.** See the worked example of Section 4.7.
4. **Tuning on the test set.** Every peek costs generalisation. Keep a locked holdout that is touched once.
5. **Ignoring the irreducible term.** Chasing error below σ^2 produces elaborate models that fit noise.
6. **Treating clustering output as truth.** Clusters are hypotheses. Validate them for stability and for utility.
7. **Forgetting to scale where scale matters.** Distance based and gradient based methods need standardised inputs; trees do not.
8. **Reporting a single number without an interval.** Repeat the fit under different seeds and folds and report a mean with a standard deviation.

7.3 The no free lunch principle

Averaged uniformly over all possible target functions, every learning algorithm has identical expected out of sample performance. There is therefore no universally best model, and any algorithm's success is a statement about the match between its inductive bias and the structure

of the particular problem. Trees assume axis aligned structure; linear models assume additivity; convolutional networks assume translation equivariance and locality; k means assumes spherical, comparably sized groups. Choosing a model *is* choosing an assumption, and the practitioner's real skill lies in matching assumption to domain.

8 Summary

1. Artificial intelligence names a goal, machine learning names a mechanism, and deep learning names a class of mechanisms.
2. Every model is a hypothesis space, a loss, an optimiser and a generalisation control. Nothing else.
3. Learning is the substitution of empirical risk, Equation (2), for the true risk, Equation (1). Everything difficult in the field stems from that substitution.
4. Paradigms are distinguished by the supervision signal, never by the algorithm. Apply the four question test of Section 3.3.
5. Supervised methods estimate $\mathbb{P}(y \mid \mathbf{x})$ and are evaluated against ground truth. Unsupervised methods characterise $\mathbb{P}(\mathbf{x})$ and are evaluated by stability and by downstream utility.
6. The bias variance identity, Equation (4), explains regularisation, ensembling and early stopping in one line each.
7. Metrics are decisions. Under imbalance, accuracy misleads; state the operating point together with recall, precision and the false positive rate.
8. There is no best algorithm, only an algorithm whose assumptions match the data in front of you.

References

- [1] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw Hill, 1997.
- [2] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [4] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2022.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [6] V. N. Vapnik, *The Nature of Statistical Learning Theory*, 2nd ed. New York, NY, USA: Springer, 2000.
- [7] S. Shalev Shwartz and S. Ben David, *Understanding Machine Learning: From Theory to Algorithms*. Cambridge, UK: Cambridge University Press, 2014.
- [8] J. R. Quinlan, *C4.5: Programs for Machine Learning*. San Mateo, CA, USA: Morgan Kaufmann, 1993.

- [9] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, 2001.
- [10] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Transactions on Information Theory*, vol. 28, no. 2, 1982.
- [11] A. P. Dempster, N. M. Laird, and D. B. Rubin, “Maximum likelihood from incomplete data via the EM algorithm,” *Journal of the Royal Statistical Society, Series B*, vol. 39, no. 1, 1977.
- [12] D. H. Wolpert, “The lack of a priori distinctions between learning algorithms,” *Neural Computation*, vol. 8, no. 7, 1996.